📚 جزوه سیستم عامل | فصل ۹: حافظه اصلی

📝 بخش ۹a۳: صفحه‌بندی (Paging) - مفاهیم پایه

۱. مقدمه: چرا صفحه‌بندی؟ 💡

در روش قبلی (تخصیص همجوار)، مجبور بودیم کل برنامه را پشت‌سرهم در حافظه بریزیم. این باعث "پارگی خارجی" می‌شد. **صفحه‌بندی (Paging)** آمد تا این قانون را بشکند:

"اجازه دهید فضای آدرس فیزیکی پردازه، غیرهمجوار (Non-contiguous) باشد."

یعنی تکه‌های برنامه شما می‌توانند هر جای حافظه پخش و پلا باشند، ولی پردازنده (CPU) طوری با آن‌ها کار می‌کند که انگار پشت‌سرهم هستند.

۲. روش پایه (Basic Method) 🗂️

در این روش، حافظه را به تکه‌های ثابت و هم‌اندازه تقسیم می‌کنیم:

الف) قاب (Frame) - در حافظه فیزیکی

- حافظه فیزیکی (RAM) را به بلوک‌های هم‌اندازه به نام **قاب (Frame)** تقسیم می‌کنیم.
- مثلاً تمام رم را به تکه‌های ۴ کیلوبایتی تقسیم می‌کنیم.

ب) صفحه (Page) - در حافظه منطقی

حافظه منطقی (برنامه) را هم به بلوک‌هایی با **همان اندازه** تقسیم می‌کنیم که به آن‌ها **صفحه (Page)** می‌گوییم.

نحوه اجرا: وقتی برنامه‌ای می‌خواهد اجرا شود، صفحاتش (Pages) برداشته می‌شوند و در هر **قاب آزاد (Free Frame)** که پیدا شد، قرار می‌گیرند (لازم نیست قاب‌ها کنار هم باشند).

۳. ترجمه آدرس (Address Translation) 🧠

وقتی CPU یک آدرس تولید می‌کند، این آدرس به دو بخش تقسیم می‌شود:

شماره صفحه (Page Number - p):

- مثل "شماره فصل" در یک کتاب.

- این عدد به عنوان ایندکس (Index) برای جستجو در **جدول صفحه (Page Table)** استفاده می‌شود.

افست صفحه (Page Offset - d):

- مثل "شماره خط" در آن فصل.
- مکان دقیق کلمه را درون آن صفحه نشان می‌دهد.
- این عدد تغییر نمی‌کند و مستقیماً به آدرس فیزیکی منتقل می‌شود.

جدول صفحه (Page Table) چیست؟

- قلب تپنده صفحه‌بندی است.
- یک جدول که می‌گوید: **"صفحه شماره X در قاب شماره Y قرار دارد."**
- آدرس پایه (Base Address) هر صفحه را در حافظه فیزیکی نگه می‌دارد.

فرآیند ترجمه توسط MMU:

1. CPU آدرس منطقی (p, d) را می‌دهد.
2. سیستم به Page Table می‌رود و خانه شماره p را چک می‌کند.
3. از آنجا شماره قاب f را پیدا می‌کند.
4. آدرس فیزیکی را با ترکیب f و d می‌سازد: (f, d).

۴. اندازه صفحه (Page Size) 📏

- اندازه صفحه همیشه **توانی از ۲** است (مثلاً ۴ کیلوبایت یا ۸ کیلوبایت).
- چرا توان ۲؟ چون تبدیل آدرس منطقی به (p, d) را برای سخت‌افزار بسیار آسان می‌کند (بدون نیاز به ضرب و تقسیم، فقط با جداسازی بیت‌ها).
  - اگر اندازه صفحه n^2 بایت باشد:
  - n بیت کم‌ارزش آدرس = **افست (d)**
  - بقیه بیت‌ها = **شماره صفحه (p)**

۵. تحلیل پارگی در صفحه‌بندی (Fragmentation) 💔

الف) پارگی خارجی (External Fragmentation)

- **نداریم! ❌**

- چون هر قابی در حافظه آزاد باشد، می‌تواند به هر صفحه‌ای داده شود. هیچ فضایی به خاطر "پیوسته نبودن" هدر نمی‌رود.

ب) پارگی داخلی (Internal Fragmentation)

- **داریم!** ✅
- چون حافظه را بلوک‌بندی کرده‌ایم.
- *مثال:* اگر اندازه صفحه ۲۰۴۸ بایت باشد و برنامه ۷۲۷۶۶ بایت باشد:
  - ۳۵ صفحه کامل (۷۱۶۸۰ بایت) پر می‌شود.
  - ۱۰۸۶ بایت باقی می‌ماند که باید در صفحه ۳۶ام ریخته شود.
  - در صفحه آخر، ۹۶۲ بایت خالی می‌ماند که هدر رفته است (**پارگی داخلی**).
- *نکته:* به طور متوسط، در هر پردازه **نیم صفحه (page 0.5)** پارگی داخلی داریم.

۶. جدول قاب (Frame Table) 🗂️

سیستم‌عامل باید بداند کدام قاب‌های فیزیکی پر هستند و کدام‌ها خالی. برای این کار یک ساختمان داده به نام **جدول قاب** دارد که وضعیت تک‌تک قاب‌های فیزیکی را نگه می‌دارد (آزاد است؟ دست کدام پردازه است؟).

📊 خلاصه نکات کلیدی برای نمره ۲۰

1. **تفاوت Page و Frame:** صفحه در دید برنامه‌نویس (منطقی) است، قاب در سخت‌افزار (فیزیکی) است. اندازه هر دو **یکی** است.
2. **جدول صفحه:** وظیفه نگاشت (Mapping) منطقی به فیزیکی را دارد.
3. **پارگی:** صفحه‌بندی پارگی خارجی را حذف می‌کند، اما پارگی داخلی کمی دارد (در صفحه آخر هر پردازه).
4. **ایزولاسیون:** پردازه نمی‌تواند به حافظه‌ای که در جدول صفحه‌اش نیست دسترسی پیدا کند، پس امنیت کامل برقرار است.